01 / Qué construimos

EL DURAZNO · GUÍA DE REPASO · 22 OCT 2026

Una consulta de fechas de prueba. Esta guía resume la sesión de JavaScript y deja el código ordenado para estudiar desde el teléfono.

El recorrido

1. HTML: agregamos entrada, salida, botón y un espacio para el mensaje.

2. Conectamos script.js con el HTML usando defer.

3. Buscamos los elementos y guardamos sus referencias en constantes.

4. Escuchamos el envío del formulario sin recargar la página.

5. Validamos el orden de las fechas y calculamos las noches.

Lo que todavía no hace

No consulta una base de datos, no compara reservas ocupadas, no bloquea fechas ni confirma reservas. El mensaje muestra la duración, no disponibilidad real.

Tres responsabilidades

HTML: estructura y campos. **CSS:** apariencia y distribución. **JavaScript:** comportamiento. Más adelante, el servidor volverá a validar la disponibilidad.

02 / Conectar la página

Los identificadores deben coincidir

```
<form id="form-disponibilidad">
  <label for="entrada">Entrada</label>
  <input type="date" id="entrada"
         name="entrada" required>
  <label for="salida">Salida</label>
  <input type="date" id="salida"
         name="salida" required>
  <button type="submit">Consultar</button>
</form>
<p id="resultado-disponibilidad"
   role="status"></p>
```

type="date" abre el selector de fechas del navegador. **required** exige completar el campo. **label for** enlaza la etiqueta con el id del campo.

El párrafo empieza vacío: JavaScript escribirá allí. role="status" permite anunciar sus cambios mediante tecnologías de asistencia.

En head, una sola vez

```
<script src="script.js" defer></script>
```

src indica el archivo. **defer** permite descargarlo mientras se procesa el HTML y ejecutarlo después del análisis de la página. head y header son elementos distintos.

03 / Identificar y escuchar

Al principio de script.js:

```
const entrada =
  document.getElementById("entrada");
const salida =
  document.getElementById("salida");
const formulario = document.getElementById(
  "form-disponibilidad");
const resultado = document.getElementById(
  "resultado-disponibilidad");
```

document representa la página. **getElementById** busca un elemento por su identificador exacto. Si no lo encuentra, devuelve null.

const impide reasignar esa variable. No congela el elemento: la persona puede elegir otra fecha y el programa puede cambiar sus propiedades.

Una primera prueba

```
formulario.addEventListener(
  "submit", function (event) {
    event.preventDefault();
    resultado.textContent =
      "Buscando disponibilidad...";
  }
);
```

submit: envío del formulario. **preventDefault():** evita la navegación normal del envío. **textContent:** escribe texto en el resultado. El mensaje de prueba no inicia una búsqueda real.

La referencia a toda la sección disponibilidad no se necesita en este paso. Los nombres propios pueden estar en inglés; los textos para huéspedes siguen en español.

04 / Leer y validar

Este es el comienzo del bloque final. Reemplaza el listener de prueba; no se agrega un segundo listener.

```
formulario.addEventListener(
  "submit", function (event) {
    event.preventDefault();

    const checkIn = entrada.value;
    const checkOut = salida.value;

    if (!checkIn || !checkOut) {
      resultado.textContent =
        "Elegí las fechas de entrada y salida.";
      return;
    }

    if (checkOut <= checkIn) {
      resultado.textContent =
        "La salida debe ser posterior " +
        "a la entrada.";
      return;
    }

    // Continúa en la página siguiente.
```

.value lee el valor actual. Se lee dentro de la función para obtener las fechas de cada consulta, no solo las que existían al cargar la página.

! significa negación; **||**, "o". Si falta cualquiera de las fechas, mostramos el mensaje. El navegador suele impedir antes el envío de campos vacíos por required.

if ejecuta un bloque si se cumple una condición. **return** termina la función para no seguir calculando después de un error.

05 / Calcular las noches

Continuación dentro de la misma función:

```
    const millisecondsPerDay =
      24 * 60 * 60 * 1000;
    const nights =
      (Date.parse(checkOut) -
       Date.parse(checkIn)) /
      millisecondsPerDay;

    resultado.textContent =
      `Tu estadía es de ${nights} ` +
      `${nights === 1 ? "noche" : "noches"}. ` +
      "Todavía no comprobamos disponibilidad.";
  }
);
```

Por qué funciona

Los campos entregan fechas válidas en formato AAAA-MM-DD. Ese formato permite comparar su orden como texto cuando ambas fechas están completas.

Date.parse convierte esas cadenas de fecha en milisegundos. Para este formato de solo fecha, ambas se interpretan en UTC. Restarlas así evita que un cambio de horario local altere la cuenta de noches.

Un día tiene 24 × 60 × 60 × 1000 milisegundos. Dividir la diferencia por ese número da la cantidad de noches.

Las comillas invertidas permiten insertar valores con **${...}**. La expresión **condición ? opción1 : opción2** elige "noche" si el valor es 1 y "noches" en los demás casos.

Entrada incluida, salida libre

Del 22 al 27 son cinco noches: 22, 23, 24, 25 y 26. La salida del 27 permite una nueva entrada ese día.

06 / Probar y continuar

Pruebas rápidas

• Mismo día de entrada y salida: debe mostrar error.

• Salida anterior a la entrada: debe mostrar error.

• Del 22 al 27 del mismo mes y año: debe mostrar cinco noches.

• Del 22 al 23: debe mostrar una noche.

• Cambiar las fechas y volver a consultar: debe actualizar el resultado sin recargar.

Si la página se recarga

Revisá que script.js esté guardado y conectado una sola vez; que los id coincidan; y que todas las llaves y los paréntesis estén cerrados. Un error de sintaxis puede impedir que se registre el listener.

En la sesión faltaba cerrar el listener. Corregimos ese cierre sin agregar otro listener debajo.

Próximos pasos acordados

• Limitar las fechas desde hoy hasta doce meses adelante, calculados al abrir la página. Todavía no está implementado.

• Simular estadías ocupadas y detectar superposiciones.

• Mejorar la presentación del calendario.

• Conectar un servidor y una base de datos. El servidor será la autoridad para confirmar disponibilidad.

Para comprobar que lo entendiste

¿Qué diferencia hay entre el campo entrada y entrada.value? ¿Por qué usamos preventDefault? ¿Qué detiene return? ¿Calcular cinco noches significa que esas noches están libres?